

Трансформеры в обучении с подкреплением Дообучение больших языковых моделей

Алексей Скрынник

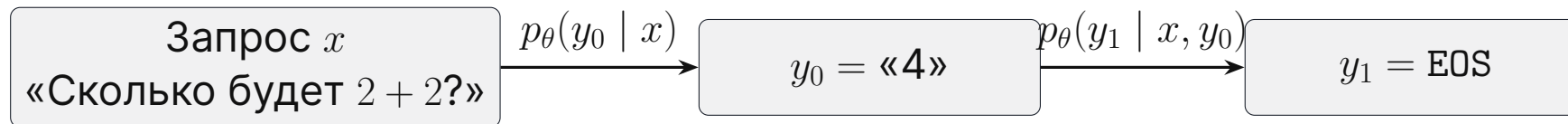
01

Авторегрессионные модели

Языковая модель: распределение над продолжениями

Словарь токенов $\mathcal{V}$; запрос $x = (x_0, \dots, x_{m-1})$; ответ $y = (y_0, \dots, y_{T-1})$, включая токен завершения EOS.

$$p_{\theta}(y \mid x) = \prod_{t=0}^{T-1} p_{\theta}(y_t \mid x, y_{<t}), \quad y_{<t} = (y_0, \dots, y_{t-1}).$$



- На каждом шаге модель выдает распределение по всему словарю $\mathcal{V}$.
- Токен может быть словом, частью слова, знаком или служебным символом.
- В примере предполагаем, что «4» занимает один токен. T зависит от ответа.

Трансформер: внимание только к доступному контексту

Для входных позиций $z = (x, y_{<t})$ строим представления H : эмбединги токенов с информацией об их позициях.

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V,$$

$$\text{Attention}(H) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

$M_{ij} = 0$ при $j \leq i$ и $-\infty$ при $j > i$.

- Запрос q_i сравнивается с ключами k_j .
- Веса внимания смешивают векторы v_j .
- Несколько голов внимания, полносвязные слои, остаточные связи и нормализация образуют блоки модели.

Какие позиции видит строка i ?

	z_0	z_1	z_2	z_3	z_4
z_0					
z_1					
z_2					
z_3					
z_4					

Доступно / Закрито маской

Представление позиции i
предсказывает токен z_{i+1} .

Обучение по тексту и генерация ответа

Обучение: известен эталонный ответ

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_{t=0}^{T-1} \log p_{\theta}(y_t \mid x, y_{<t}).$$

- На вход подаем правильный префикс из данных (*teacher forcing*).
- Сдвиг входов и целей на одну позицию; маска закрывает будущие токены.
- Потери всех позиций ответа вычисляются параллельно. Токены запроса здесь не входят в сумму.

Генерация: ответ еще неизвестен

$$y_t \sim p_{\theta}(\cdot \mid x, y_{<t}).$$

- Предсказать распределение следующего токена.
- Выбрать токен и дописать его к контексту.
- Повторять до EOS или ограничения длины.
- Префикс состоит из собственных предыдущих ответов модели.

Что оптимизируем?

Предобучение: вероятность токенов корпуса. SFT: вероятность эталонного ответа. RL: ожидаемая награда за сгенерированный ответ.

02



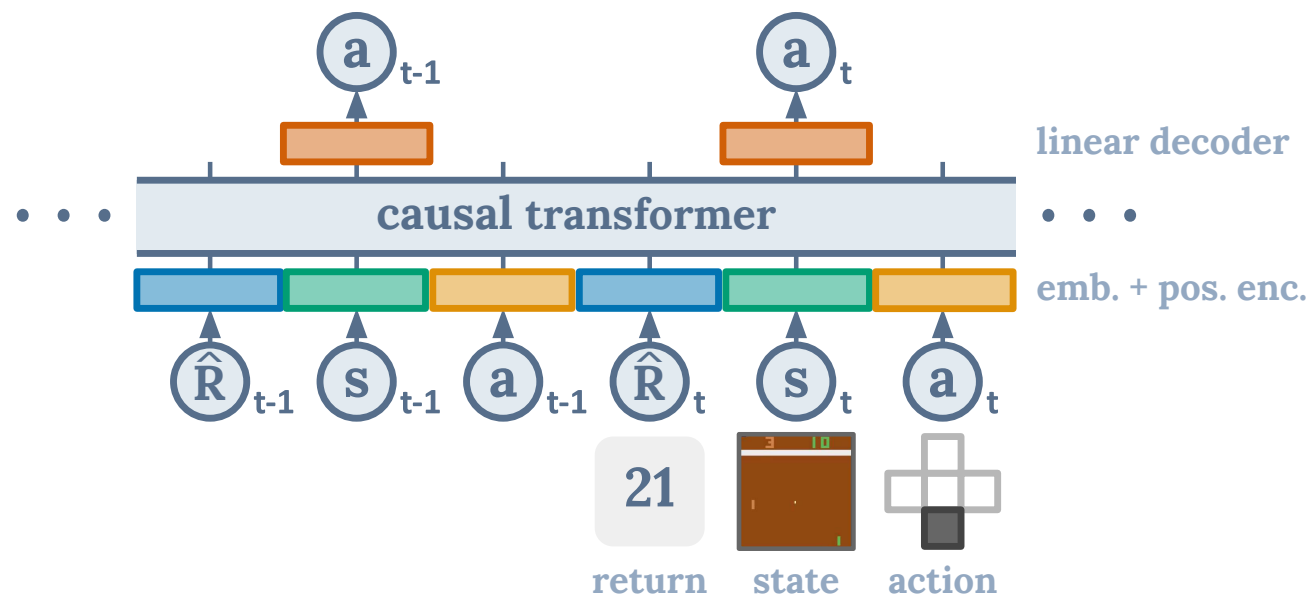
Decision Transformer

Decision Transformer: RL как последовательность

- Траектория записывается как последовательность токенов:

$$(\hat{R}_0, o_0, a_0, \hat{R}_1, o_1, a_1, \dots).$$

o_t обозначает наблюдение, a_t обозначает действие, $\hat{R}_t$ задает оставшуюся желаемую отдачу.



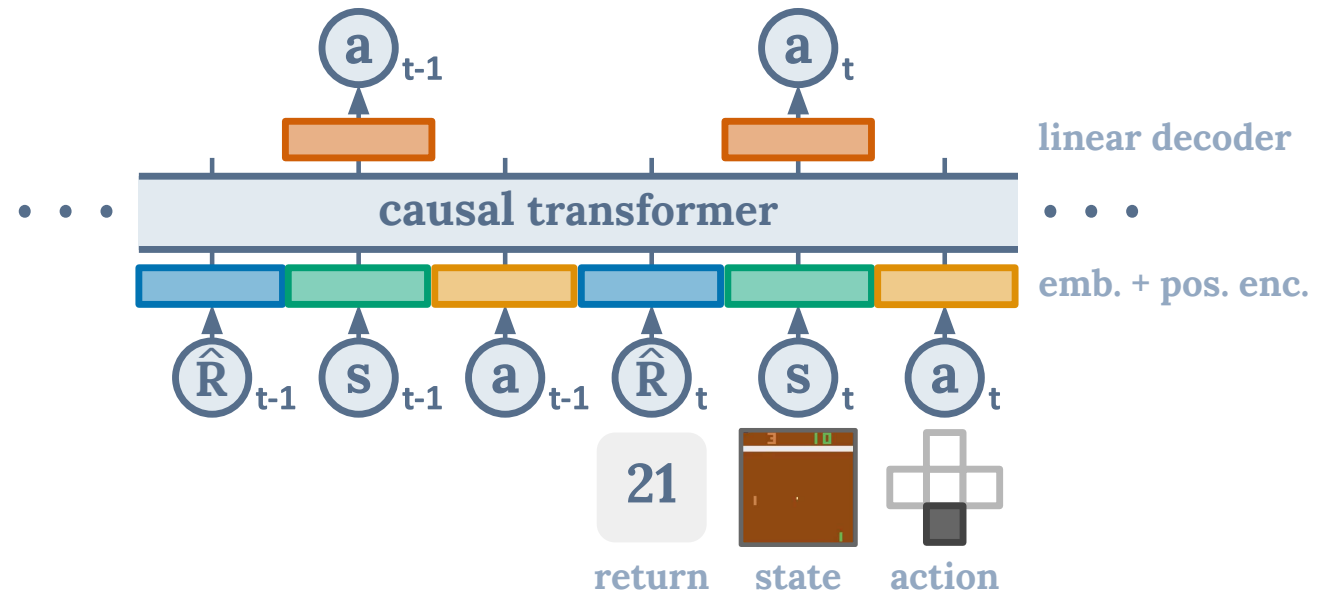
Decision Transformer: RL как последовательность

- Траектория записывается как последовательность токенов:

$$(\hat{R}_0, o_0, a_0, \hat{R}_1, o_1, a_1, \dots).$$

o_t обозначает наблюдение, a_t обозначает действие, $\hat{R}_t$ задает оставшуюся желаемую отдачу.

- Причинный трансформер предсказывает действие a_t по доступному контексту.



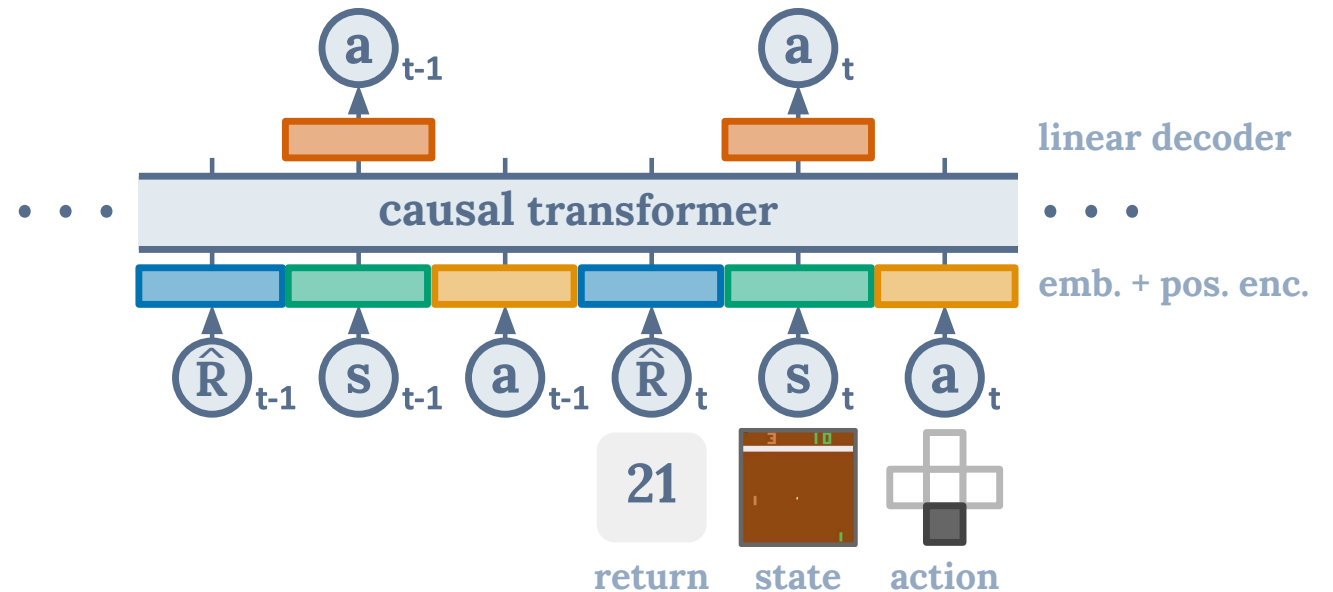
Decision Transformer: RL как последовательность

- Траектория записывается как последовательность токенов:

$$(\hat{R}_0, o_0, a_0, \hat{R}_1, o_1, a_1, \dots).$$

o_t обозначает наблюдение, a_t обозначает действие, $\hat{R}_t$ задает оставшуюся желаемую отдачу.

- Причинный трансформер предсказывает действие a_t по доступному контексту.
- Желаемая отдача становится условием для генерации поведения.



Decision Transformer: обучение и применение

$$\pi_{\theta}(a_t \mid \hat{R}_{\leq t}, o_{\leq t}, a_{< t})$$

Обучение

- обучение на фиксированном наборе траекторий
- на вход подаем префикс из данных (teacher forcing)
- предсказываем действие a_t

Decision Transformer: обучение и применение

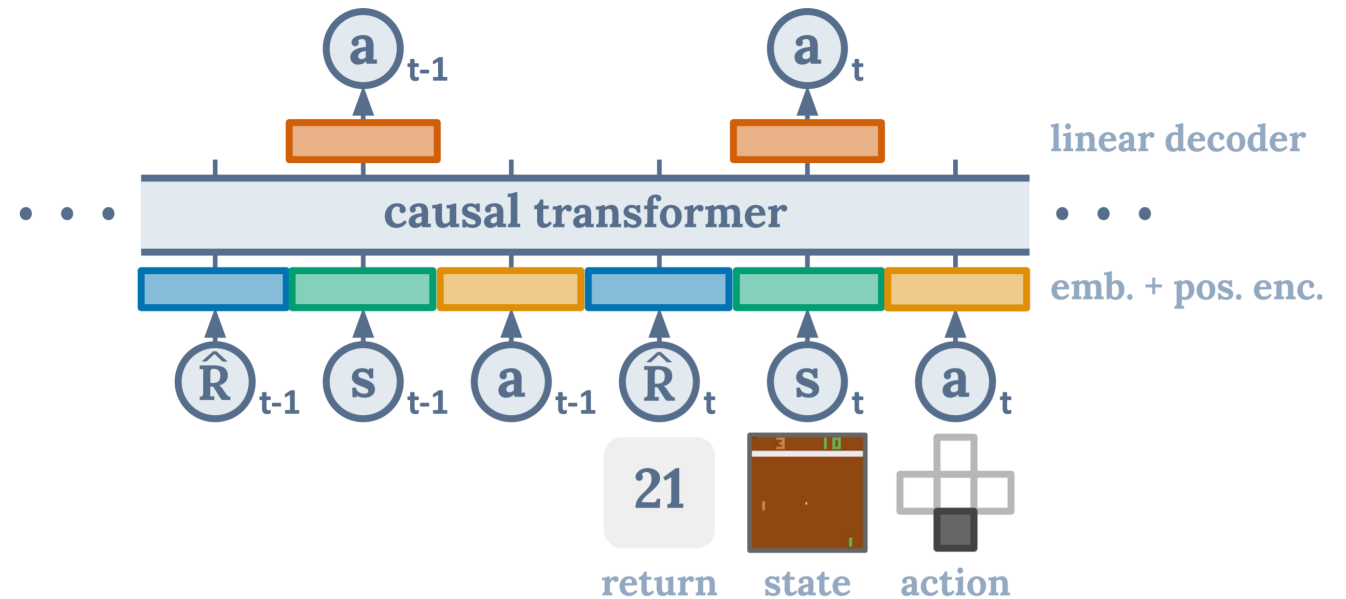
$$\pi_{\theta}(a_t \mid \hat{R}_{\leq t}, o_{\leq t}, a_{< t})$$

Обучение

- обучение на фиксированном наборе траекторий
- на вход подаем префикс из данных (teacher forcing)
- предсказываем действие a_t

Применение

- задаем желаемую отдачу $\hat{R}_0$
- наблюдаем o_t и генерируем a_t
- после награды r_t обновляем условие:
 $\hat{R}_{t+1} = \hat{R}_t - r_t$



Отдача как условие: пример с числами

Для эпизода из данных при $\gamma = 1$ вычисляем оставшуюся отдачу:

$$G_t = \sum_{k=t}^{T-1} r_k, \quad \hat{R}_t := G_t \quad \text{при подготовке обучающих примеров.}$$

Шаг t	0	1	2
Вознаграждение за действие r_t	0	1	2
Оставшаяся отдача G_t	3	3	2
Токены траектории	$(3, o_0, a_0) \quad (3, o_1, a_1) \quad (2, o_2, a_2)$		

При обучении

Будущие награды известны из записанной траектории. Превращаем достигнутую отдачу в условие для предсказания действий.

При применении

Выбираем цель $\hat{R}_0 = 3$. После награды $r_0 = 0$ остается 3; после $r_1 = 1$ остается 2: $\hat{R}_{t+1} = \hat{R}_t - r_t$.

Желаемая отдача $\hat{R}_t$ задает условие генерации.

Decision Transformer: цель обучения и ограничения

Контекст $h_t = (\hat{R}_{\leq t}, o_{\leq t}, a_{< t})$; на практике берем окно истории. Для дискретных действий:

$$\mathcal{L}_{\text{DT}}(\theta) = -\mathbb{E}_{\tau \sim \mathcal{D}_{\text{off}}} \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t \mid h_t).$$

Для непрерывных действий можно предсказывать вектор $\hat{a}_{\theta}(h_t)$ и минимизировать $\|a_t - \hat{a}_{\theta}(h_t)\|_2^2$.

- Предсказываем действия с условием на отдачу: без обновлений по уравнению Беллмана или градиенту стратегии.
- Покрываем данные ограничено: завышенная цель может оказаться вне обучающего распределения.
- При случайных наградах высокая отдача может отражать удачу среды, которую стратегия не сможет воспроизвести.

Связь с LLM: DT использует отдачу как вход. При RL-дообучении LLM вознаграждение за ответ служит сигналом для изменения стратегии.

03

Генерация текста как МППР

LLM как стратегия: состояния и действия

Рассматриваем один запрос и один ответ без внешних инструментов:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle.$$

- **Начальное состояние:** запрос $x = (x_0, \dots, x_{m-1}) \sim \mu$ из набора $\mathcal{X}$; μ задает распределение запросов, $s_0 = (x, \emptyset)$.
- **Состояние:** весь доступный контекст $s_t = (x, y_{<t})$, где $y_{<t} = (y_0, \dots, y_{t-1})$. $\mathcal{S}$ содержит такие контексты и терминальные состояния.
- **Действие:** следующий токен $a_t = y_t \in \mathcal{A} = \mathcal{V}$. Токен EOS также принадлежит словарю.
- **Стратегия:** распределение, задаваемое языковой моделью:

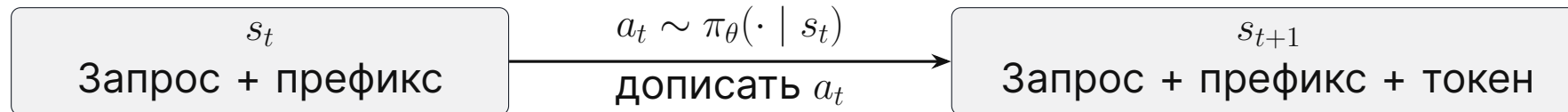
$$\pi_{\theta}(a_t \mid s_t) = p_{\theta}(y_t \mid x, y_{<t}).$$

Почему состояние марковское?

Префикс уже содержит всю историю генерации: по нему и выбранному токenu определяются следующий контекст, завершение и вознаграждение.

Переход: дописать выбранный токен

$$s_{t+1} = f(s_t, a_t) = (x, y_{\leq t}), \quad \mathcal{P}_{ss'}^a = \mathbb{I}\{s' = f(s, a)\}.$$



- Переход детерминирован; случайность возникает при выборе запроса и токенов.
- Эпизод заканчивается при $y_t = \text{EOS}$ или $t = T_{\max} - 1$. Длина $T \leq T_{\max}$ равна числу сгенерированных токенов.
- Терминальный контекст считаем поглощающим: дальнейшие вознаграждения нулевые.
- В этой постановке весь запрос и ответ помещаются в окно контекста. Ограничение длины известно из длины префикса.

Вероятность ответа: $\pi_\theta(y \mid x) = \prod_{t=0}^{T-1} \pi_\theta(y_t \mid x, y_{<t}).$

Вознаграждение, отдача и цель дообучения

Вознаграждение r_t получаем за токен $a_t = y_t$, $t = 0, \dots, T - 1$.

$$r_t = \begin{cases} 0, & t < T - 1, \\ R(x, y), & t = T - 1, \end{cases} \quad \mathcal{R}_s^a = \mathbb{E}[r_t \mid s_t = s, a_t = a].$$

$R(x, y) \in \mathbb{R}$ — награда за полный ответ: $r_\psi(x, y)$ или результат проверки. Правило оценивания определено и для ответов, оборванных по $T_{\max}$.

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k, \quad \gamma = 1 \Rightarrow G_t = R(x, y).$$

$$\boxed{\max_{\theta} J(\theta) = \mathbb{E}_{x \sim \mu, y \sim \pi_{\theta}(\cdot|x)}[R(x, y)]}$$

- При $\gamma = 1$ длина ответа сама по себе не дисконтирует итоговую награду.
- При SFT подражаем эталону; при RL повышаем ожидаемую отдачу собственных ответов.
- Это базовая цель; регуляризацию относительно опорной стратегии добавим позже.

Генерация ответа как траектория

Пример: $x = \text{«Сколько будет } 2 + 2?»$, $y = (\text{«4»}, \text{EOS})$. Проверка дает $R(x, y) = 1$ за правильный ответ и 0 за неправильный.

Шаг	Состояние s_t	Действие a_t	$\pi_\theta(a_t \mid s_t)$	r_t
0	$(x, \emptyset)$	«4»	0.8	0
1	$(x, \text{«4»})$	EOS	0.75	1

$\pi_\theta(y \mid x) = 0.8 \cdot 0.75 = 0.6$, $G_0 = G_1 = 1$ ($\gamma = 1$).

- Вероятности условные, заданы для примера; остальные токены имеют оставшуюся массу.
- Вероятность 0.6 относится к ответу («4», EOS), а не всех возможных правильных формулировок.
- Один итоговый результат оценивает целую последовательность действий. Как распределить этот сигнал между токенами? К этому вернемся в PPO и GRPO.

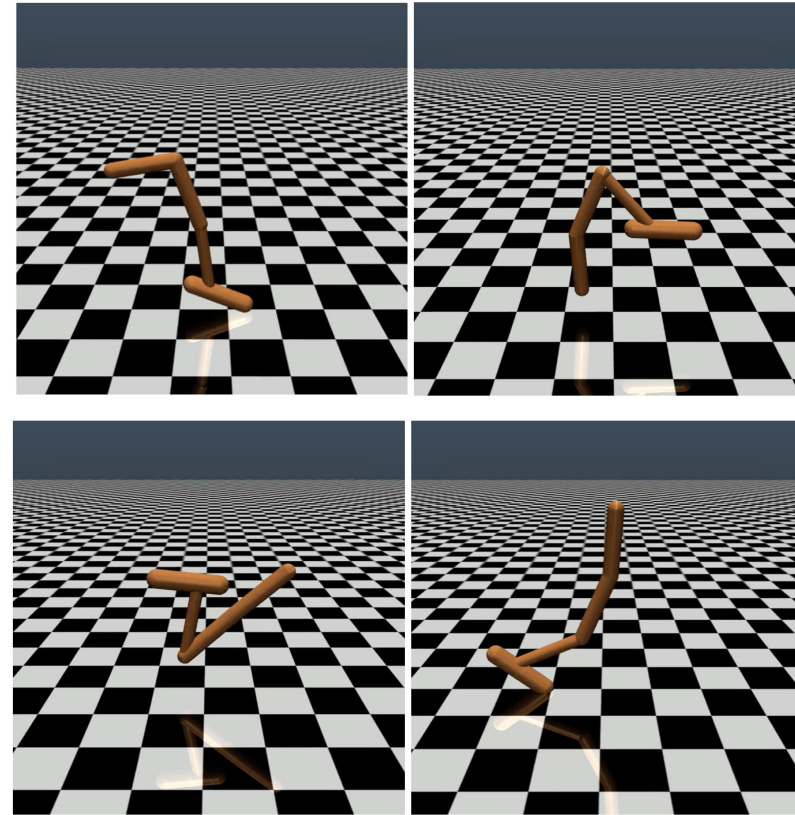
04



RLHF

Обучение по предпочтениям: от движений к тексту

- **2017:** Hopper учится делать сальто по сравнениям коротких видео.
- **900 сравнений**, менее часа работы разметчика.
- Позже сравнивают тексты: пересказы (2020), ответы InstructGPT (2022).

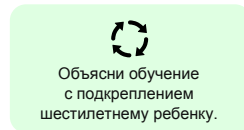


RLHF: схема обучения ChatGPT

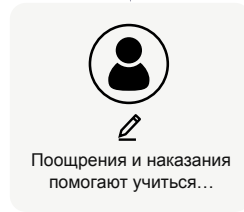
Шаг 1

**Сбор демонстраций
и обучение стратегии**

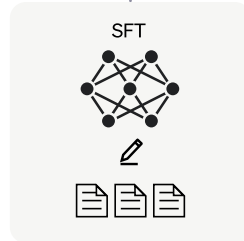
Выбираем запрос
из набора данных.



Разметчик
составляет
эталонный ответ.



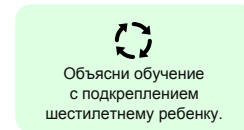
Дообучаем GPT-3.5
на демонстрациях
с учителем.



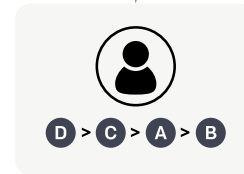
Шаг 2

**Сбор сравнений и обучение
модели вознаграждения**

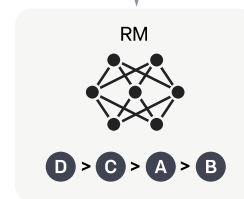
Для одного запроса
генерируем
несколько ответов.



Разметчик
упорядочивает
ответы по качеству.



Обучаем модель
вознаграждения
на сравнениях.



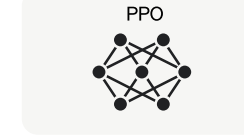
Шаг 3

**Дообучение стратегии
по модели вознаграждения
с помощью PPO**

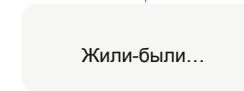
Выбираем новый
запрос из данных.



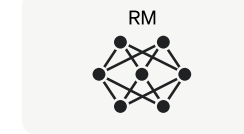
Инициализируем
стратегию PPO
из модели SFT.



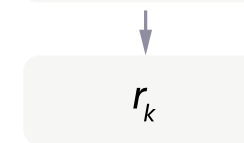
Стратегия
генерирует ответ.



Модель оценивает
вознаграждение
за ответ.



Обновляем
стратегию PPO.



Шаг 1: обучение по демонстрациям

1 / SFT → 2 / Предпочтения → 3 / PPO

Для запроса x разметчик составляет эталонный ответ y^* .

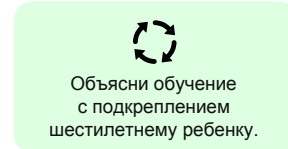
Дообучаем предобученную модель предсказывать токены эталонного ответа:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x, y^*) \sim \mathcal{D}_{\text{demo}}} \sum_t \log \pi_{\theta}(y_t^* \mid x, y_{<t}^*).$$

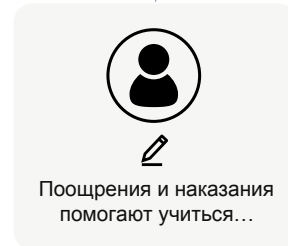
Результат: начальная стратегия π_{SFT} .

Это клонирование поведения для языковой модели: токены эталонного ответа задают целевые действия.

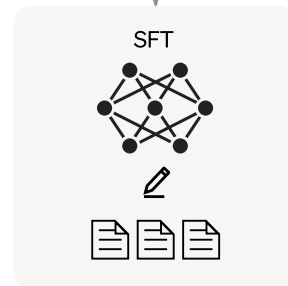
Выбираем запрос из набора данных.



Разметчик составляет эталонный ответ.



Дообучаем GPT-3.5 на демонстрациях с учителем.



Сравнение ответов: критерии предпочтения

Запрос: «Объясни коротко, почему модель хорошо работает на обучающих данных, но плохо на новых».

Ответ А

Модель слишком подстроилась под обучающие примеры и их случайные особенности, поэтому плохо обобщает на новые данные.

Ответ В

Модель запомнила обучающие примеры, но не научилась находить общие закономерности.

Какой ответ предпочтительнее?

Критерии: корректность, понятность, краткость.

До сбора предпочтений нужно договориться о критериях.

Шаг 2а: сбор предпочтений

2 / Предпочтения: сбор сравнений

Генерируем несколько ответов на **один и тот же запрос**. Разметчик упорядочивает их по качеству.

Из ранжирования на схеме получаем шесть пар сравнений:

$$D \succ C \succ A \succ B$$
$$(D, C), (D, A), (D, B), (C, A), (C, B), (A, B).$$

Результат: тройки (x, y^+, y^-) :

y^+ : предпочтительный ответ в паре; y^- : менее предпочтительный.

Разметчик задает порядок ответов, а не численные значения вознаграждения.

Для одного запроса генерируем несколько ответов.

Разметчик упорядочивает ответы по качеству.



Шаг 26: обучение модели вознаграждения

2 / Предпочтения: обучение модели вознаграждения

Оцениваем оба ответа **одной и той же моделью**:

$$\Delta r_\psi = r_\psi(x, y^+) - r_\psi(x, y^-).$$

Моделируем вероятность предпочтения:

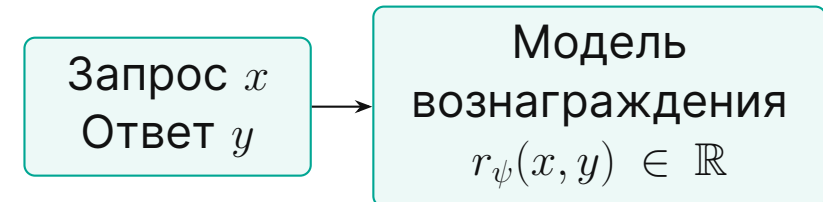
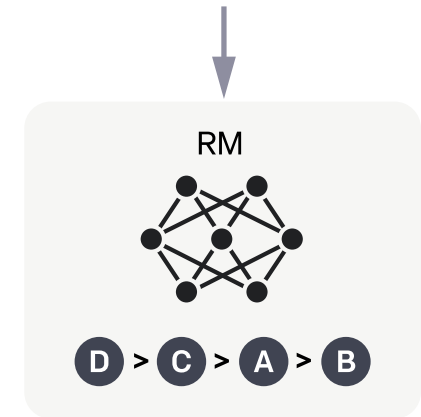
$$P_\psi(y^+ \succ y^- \mid x) = \sigma(\Delta r_\psi).$$

Обучаемся на сравнениях разметчиков:

$$\mathcal{L}_{\text{RM}}(\psi) = -\mathbb{E}_{\mathcal{D}_{\text{pref}}} \log \sigma(\Delta r_\psi).$$

Результат: модель оценивает новые ответы.
На время RL-дообучения ее параметры фиксированы.

Обучаем модель
вознаграждения
на сравнениях.



Шаг 3: дообучение стратегии с помощью PPO

3 / PPO: генерация → оценка → обновление

Инициализируем $\pi_\theta \leftarrow \pi_{\text{SFT}}$. Для каждого запроса генерируем ответ и оцениваем его фиксированной моделью вознаграждения.

Постановка RL

Состояние: запрос и сгенерированный префикс.

Действие: следующий токен. Эпизод: один ответ.

Повышаем вознаграждение, ограничивая отклонение от опорной стратегии:

$$\max_{\theta} \mathbb{E}_{x \sim \mu, y \sim \pi_{\theta}(\cdot | x)} \left[r_{\psi}(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{SFT}}(y | x)} \right].$$

Ожидание логарифма отношения дает KL-штраф. PPO обновляет стратегию по собранным траекториям; модель вознаграждения и опорная стратегия SFT фиксированы.

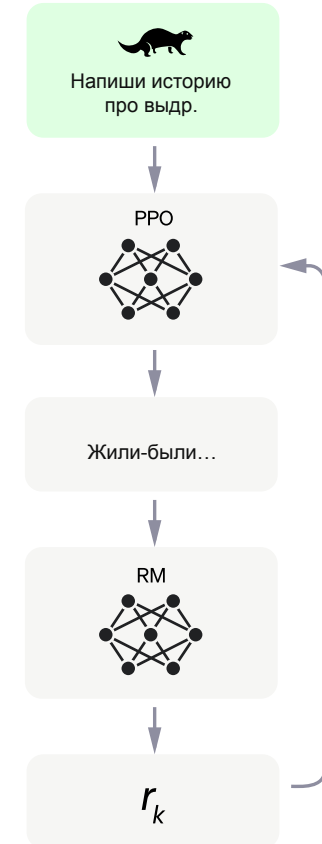
Выбираем новый запрос из данных.

Инициализируем стратегию PPO из модели SFT.

Стратегия генерирует ответ.

Модель оценивает вознаграждение за ответ.

Обновляем стратегию PPO.



Модель вознаграждения и критик

Модель вознаграждения

Вход: запрос и **готовый ответ**.

$$r_{\psi}(x, y) \in \mathbb{R}.$$

«Насколько хорош этот ответ?»

Фиксирована на этапе RL.

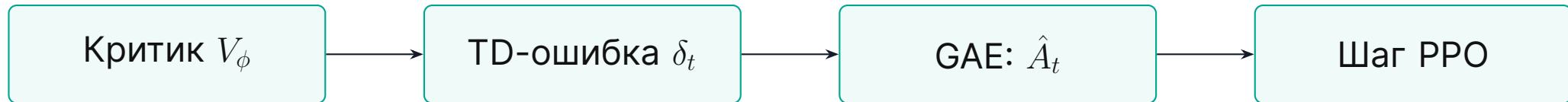
Критик

Вход: запрос и **префикс ответа**.

$$V_{\phi}(x, y_{<t}) \approx \mathbb{E}[G_t \mid x, y_{<t}].$$

«Какую отдачу еще ожидаем?»

Обучается на траекториях стратегии.



Та же схема оценки преимущества; $s_t = (x, y_{<t})$.

Reward hacking: пример CoastRunners

- **Задача:** пройти гонку.
- **Награда:** очки за попадания по мишеням.
- **Reward hacking:** агент собирает мишени по кругу, набирая очки без завершения гонки.

В RLHF: стратегия может использовать ошибки r_ψ , повышая оценку без улучшения ответа.

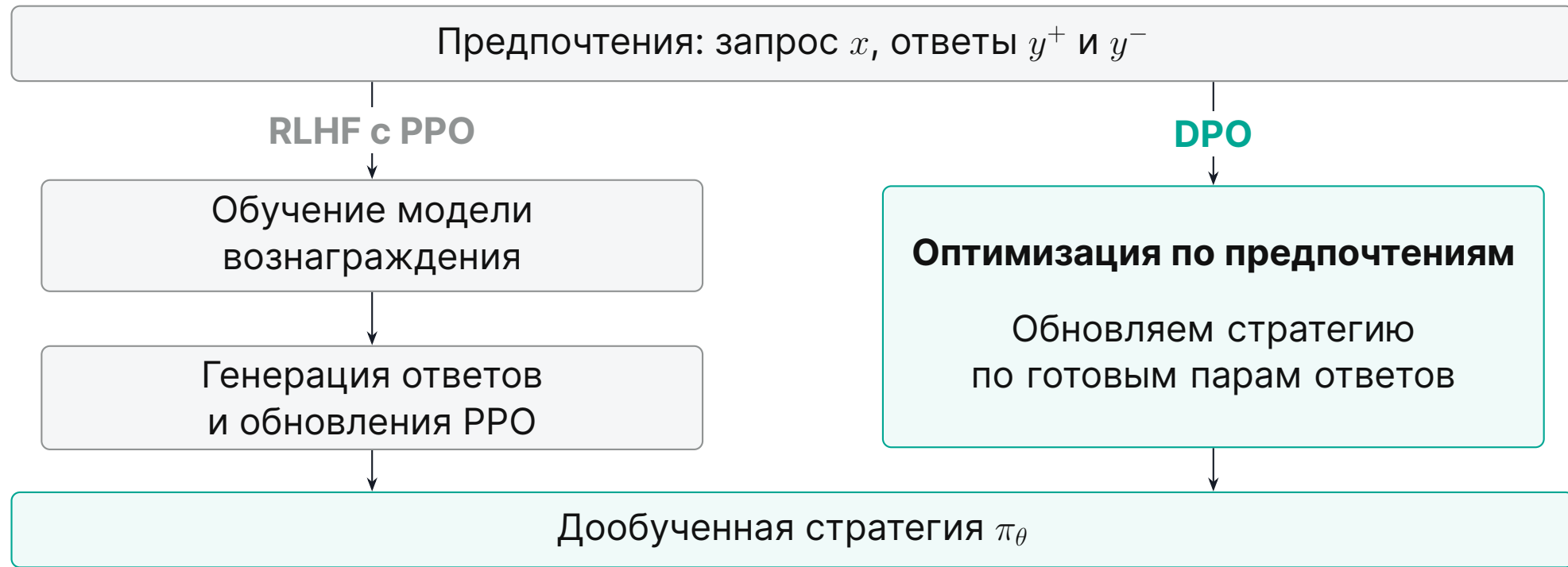


05



DPO

DPO: обучение стратегии по предпочтениям



DPO: предпочтение зависит от запроса

Запрос: «Объясни новичку, что такое переобучение. Без формул и сложных терминов».

y^+ : понятное объяснение

Модель слишком подстраивается под примеры, на которых училась. Поэтому на новых примерах она ошибается чаще.

y^- : техническое определение

При переобучении происходит чрезмерная подгонка модели под обучающую выборку, сопровождающаяся ухудшением обобщающей способности.

- Оба ответа корректны, но y^+ лучше соответствует запросу.
- Разметчик предпочитает доступное объяснение: $y^+ \succ y^-$.
- DPO усиливает относительное предпочтение y^+ перед y^- .

DPO: сравниваем с исходной моделью

- Фиксируем опорную стратегию $\pi_{\text{ref}} = \pi_{\text{SFT}}$.
- Для каждого ответа измеряем изменение вероятности:

$$d_{\theta}(x, y) = \log \frac{\pi_{\theta}(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

- DPO поощряет положительную разность:

$$d_{\theta}(x, y^{+}) - d_{\theta}(x, y^{-}) > 0.$$

Изменения для пары ответов

Ответ	$\pi_{\theta}/\pi_{\text{ref}}$	d_{θ}
y^{+} : понятный	2	$\log 2$
y^{-} : технический	1	0

Относительное предпочтение
усилилось вдвое

DPO: функция потерь

- Разность изменений задает оценку пары:

$$m_{\theta} = \beta [d_{\theta}(x, y^{+}) - d_{\theta}(x, y^{-})].$$

- Функция потерь для одной пары:

$$\ell_{\text{DPO}} = -\log \sigma(m_{\theta}), \quad \sigma(m) = \frac{1}{1 + e^{-m}}.$$

- Чем выше оценка пары, тем меньше потери.

$\beta > 0$ задает масштаб и связан с KL-регуляризацией. Для батча $\mathcal{B}$: $\mathcal{L}_{\text{DPO}} = |\mathcal{B}|^{-1} \sum_{i \in \mathcal{B}} \ell_{\text{DPO}}^{(i)}$.

Пара объяснений переобучения

y^{+} : понятное; y^{-} : техническое.

Берем $\beta = 1$.

$\pi_{\theta} = \pi_{\text{ref}}$ После изменения

m_{θ}	0	$\log 2$
$\sigma(m_{\theta})$	1/2	2/3
ℓ_{DPO}	0.693	0.405

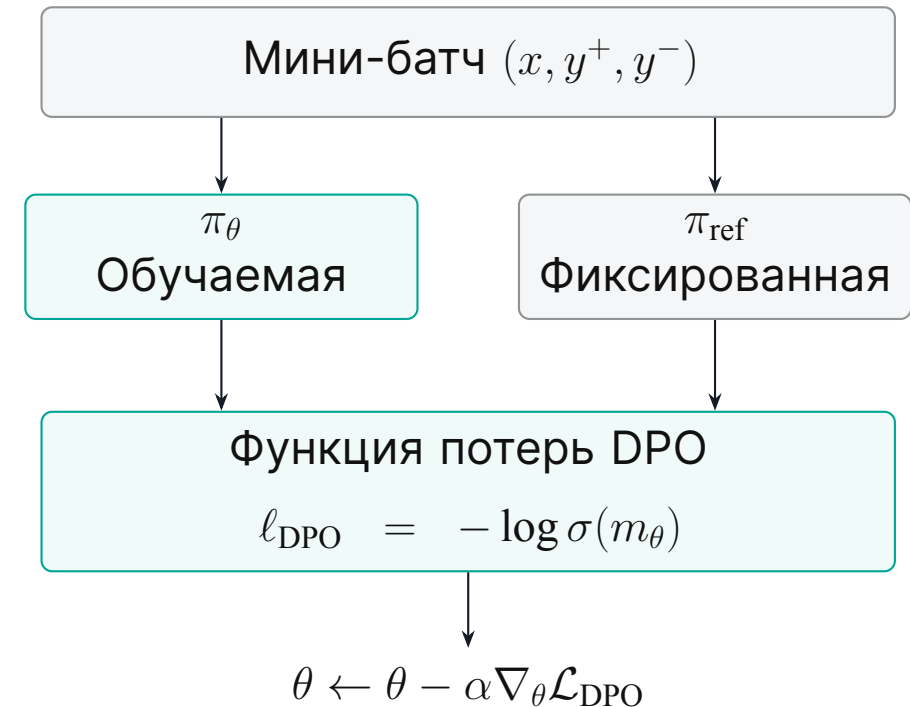
Вероятность y^{+} выросла вдвое, вероятность y^{-} не изменилась.

DPO: один шаг обучения

Инициализируем $\pi_\theta \leftarrow \pi_{\text{ref}}$.

1. Берем мини-батч (x, y^+, y^-) из данных.
2. Оцениваем оба ответа обучаемой и исходной моделями.
3. Вычисляем изменения d_θ , оценку пары m_θ и потери.
4. Обновляем только параметры θ .

Ответы берем из данных. На этом шаге модель оценивает готовый текст, а не генерирует новый. Вероятность ответа считаем по всем его токенам. Отдельную модель вознаграждения и критик не обучаем.



06

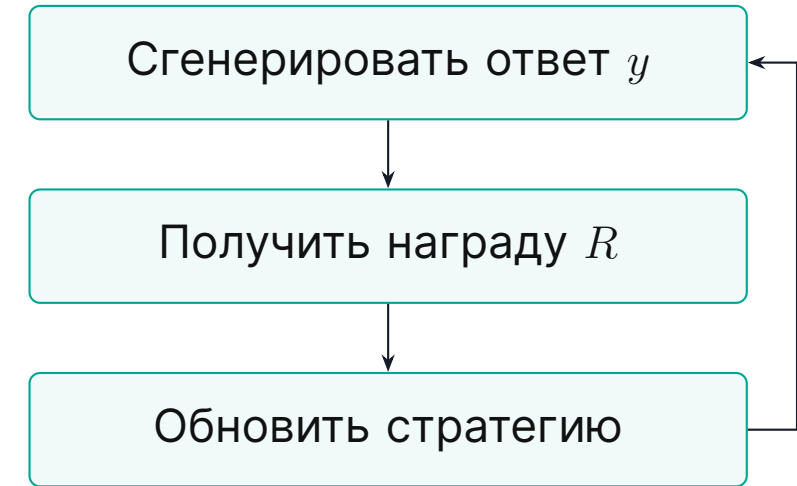


REINFORCE

REINFORCE для языковой модели

- Ответ можно оценить: тестами, проверкой решения или моделью вознаграждения.
- Хотим повышать награду **собственных ответов** текущей стратегии.
- После генерации известна полная отдача: используем ее для обновления стратегии **без критика**.

$\gamma = 1$; награда только в конце; без KL-штрафа.



Градиент стратегии для последовательности токенов

Монте-Карло оценка градиента стратегии

$$\hat{g} = \frac{1}{B} \sum_{i=1}^B \sum_{t=0}^{T^{(i)}-1} G_t^{(i)} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}).$$

Подставляем: $R_i = R(x^{(i)}, y^{(i)})$, $G_t^{(i)} = R_i$, $a_t^{(i)} = y_t^{(i)}$, $s_t^{(i)} = (x^{(i)}, y_{<t}^{(i)})$.

$$\hat{g} = \frac{1}{B} \sum_{i=1}^B R_i \sum_{t=0}^{T^{(i)}-1} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)})$$

Одна награда взвешивает градиенты всех токенов ответа.

Ответы сэмплируем из **текущей** стратегии.

Базовый уровень без критика

Для действия: $G_t - b(s_t)$ $\rightarrow$ **Для ответа:** $w = R(x, y) - b(x)$.

$b(x)$ не зависит от выбранного ответа: средний градиент сохраняется.

«Сколько будет $2 + 2$?» Заранее задали $b = 0.5$.

Ответ	Награда	Вес градиента
«4», EOS	1	+0.5
«5», EOS	0	-0.5

Базовый уровень не обязательно обучать нейросетью.

Можно начать с $b = 0$ или статистики прошлых батчей.

REINFORCE: зачем используют в языковых моделях

- **Простота:** обновляем стратегию по награде полного ответа.
- **Экономия памяти:** не храним и не обучаем критик.
- **Гибкая оценка:** модель вознаграждения или автоматическая проверка.

Разброс наград делает обновления шумными. В RLOO базовый уровень оцениваем по нескольким ответам на один запрос.

RLHF

Диалоги и краткие пересказы
Оценка моделью вознаграждения

Проверяемые задачи

Математика и код
Проверка ответа или тесты

RLOO: базовый уровень из других ответов

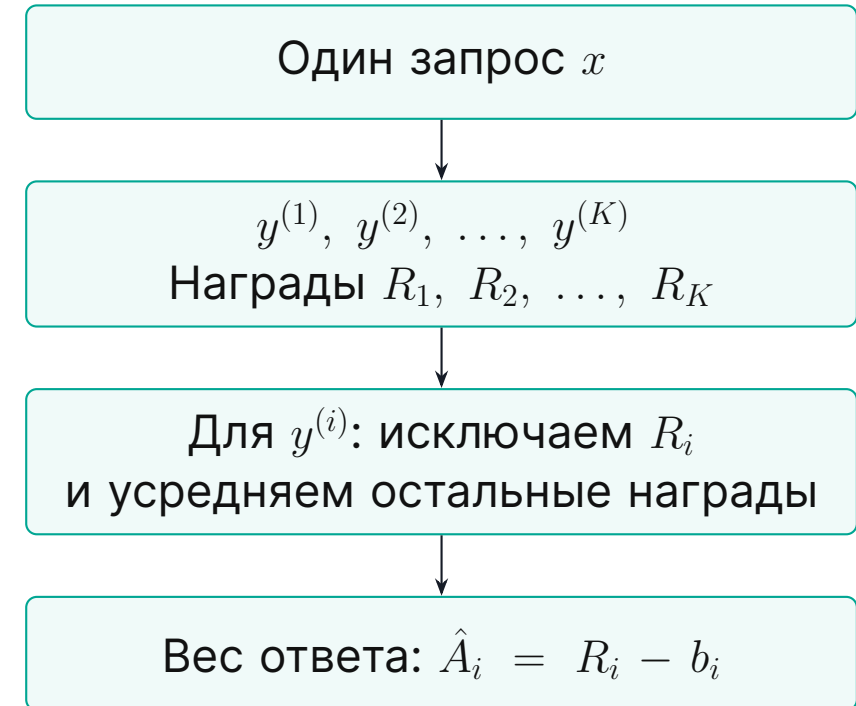
REINFORCE Leave-One-Out (RLOO)

- На один запрос независимо генерируем $K \geq 2$ ответов текущей стратегией.
- Для ответа i базовый уровень — средняя награда **остальных** ответов.

$$b_i = \frac{1}{K-1} \sum_{j \neq i} R_j, \quad \hat{A}_i = R_i - b_i.$$

Используем $\hat{A}_i$ как вес в REINFORCE; $R_i = R(x, y^{(i)})$.

При независимом сэмплировании b_i не зависит от ответа $y^{(i)}$: вычитание baseline сохраняет средний градиент.



07

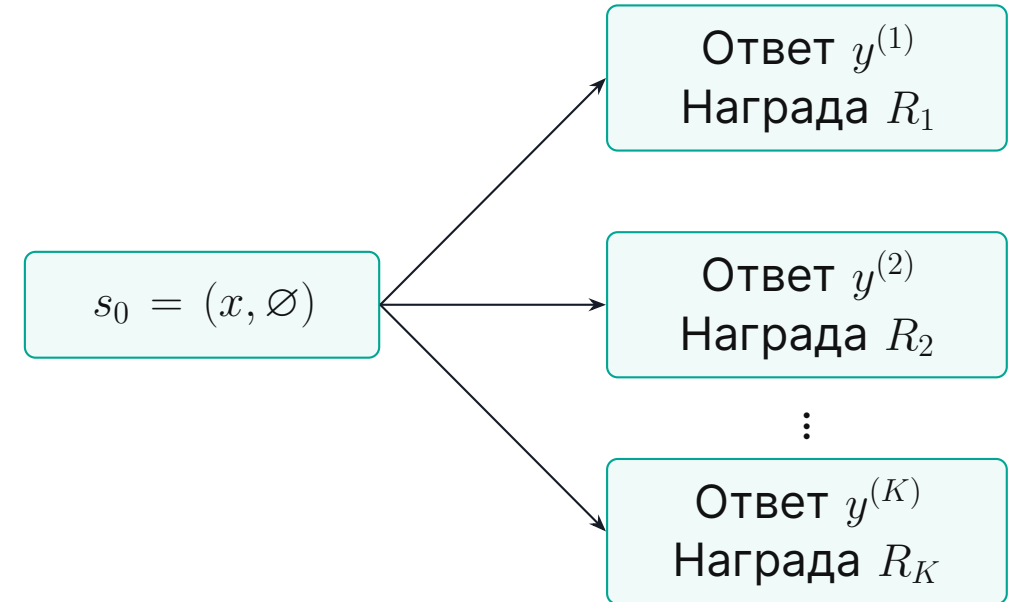


GRPO

Повторные запуски из одного состояния

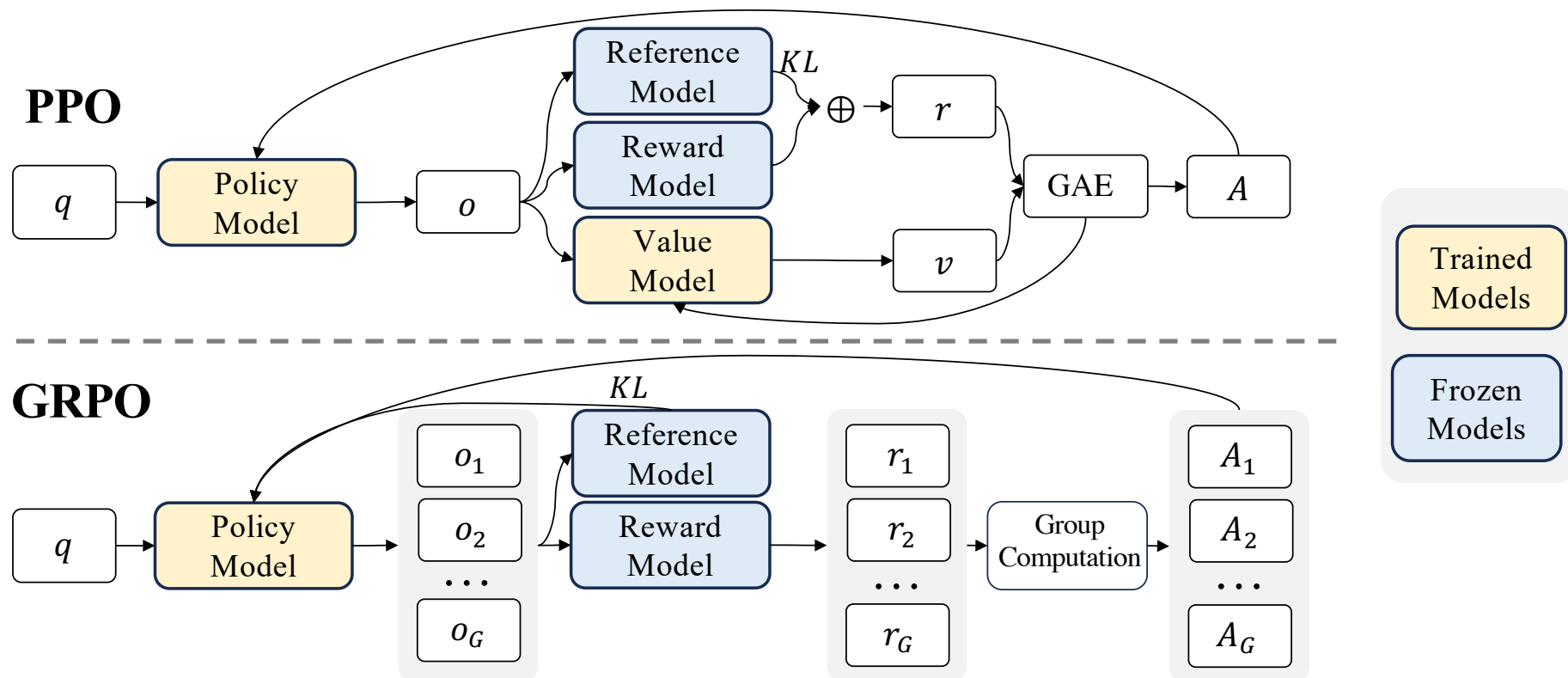
- Выбираем запрос x и задаем начальное состояние $s_0 = (x, \emptyset)$.
- Повторяем генерацию из того же контекста с новым сэмплированием токенов.
- Сравниваем результаты для одной задачи, фиксируя ее сложность.

$$y^{(i)} \sim \pi^{\text{old}}(\cdot \mid x), \quad i = 1, \dots, K.$$



Группа ответов позволяет оценить результат относительно других продолжений из s_0 .

GRPO: групповая оценка вместо критика



➤ Преимущество вычисляем по наградам группы, без обучения критика.

От наград к относительным преимуществам

- Для одного запроса получаем награды $R_i = R(x, y^{(i)})$, $i = 1, \dots, K$.
- Центрируем и нормируем их внутри группы; $\epsilon_{\text{num}} > 0$ защищает от деления на ноль.

$$\bar{R} = \frac{1}{K} \sum_{i=1}^K R_i, \quad \sigma_R = \sqrt{\frac{1}{K} \sum_{i=1}^K (R_i - \bar{R})^2}.$$

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \bar{R}}{\sigma_R + \epsilon_{\text{num}}}$$

- При итоговой награде все токены ответа получают один вес.
- В отличие от RLOO, среднее включает R_i ; добавляется нормировка по разбросу.

Пример: одна задача, четыре решения

Запрос: «Напиши функцию, которая удаляет повторы, сохраняя порядок элементов».

Награда равна доле пройденных тестов.

Решение	Тесты	R_i	$\hat{A}_i$
$y^{(1)}$	4/4	1	$+\sqrt{2}$
$y^{(2)}$	2/4	0.5	0
$y^{(3)}$	0/4	0	$-\sqrt{2}$
$y^{(4)}$	2/4	0.5	0

$$\bar{R} = 0.5, \quad \sigma_R = \sqrt{0.125} \approx 0.354 \quad (\epsilon_{\text{num}} = 0).$$

- $y^{(1)}$ получает положительный вес, $y^{(3)}$ получает отрицательный.
- $y^{(2)}$ и $y^{(4)}$ не дают вклада в градиент награды, хотя прошли часть тестов.

Обновление стратегии с ограничением шага

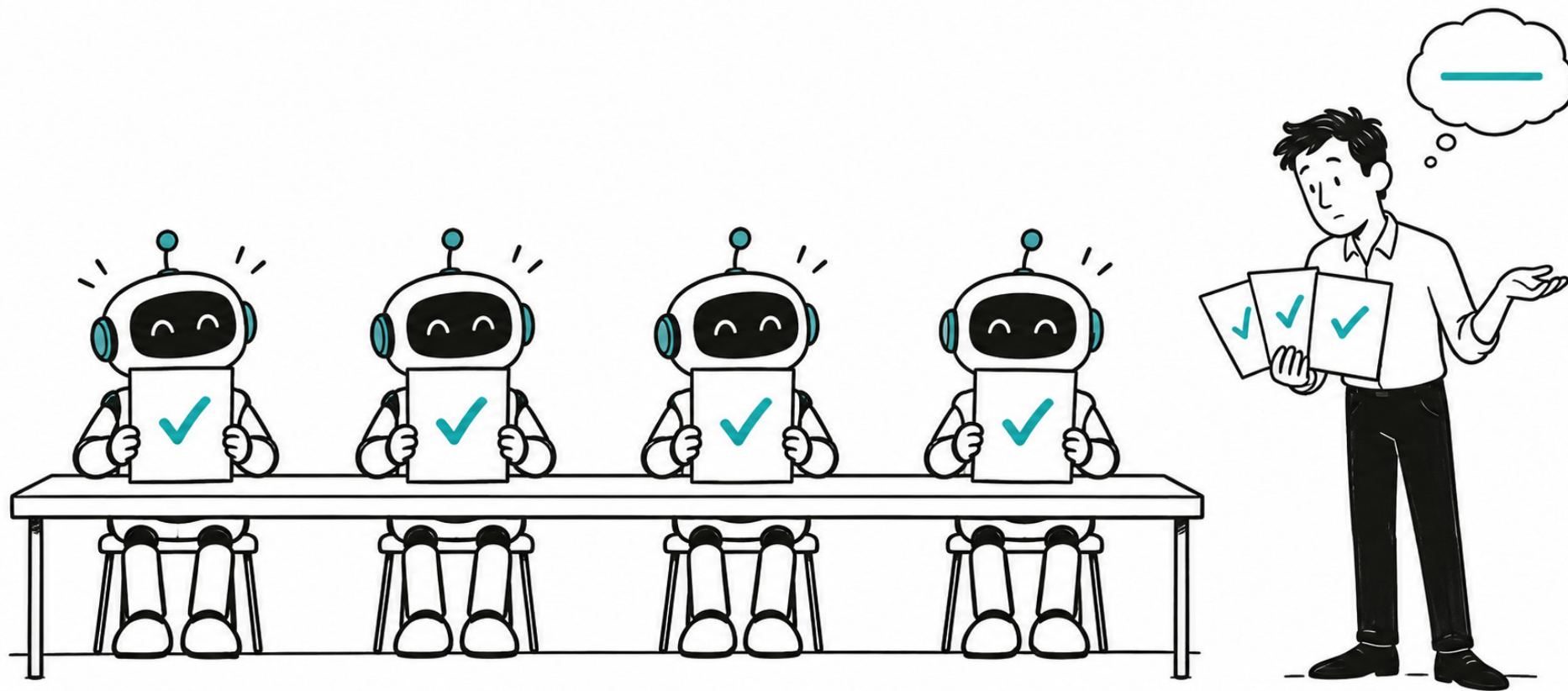
- Ответы генерирует π^{old} ; обновляем π_θ .
- Подставляем групповое преимущество в ограниченную цель PPO:

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(y_t^{(i)} \mid x, y_{<t}^{(i)})}{\pi^{\text{old}}(y_t^{(i)} \mid x, y_{<t}^{(i)})},$$

$$J_{i,t}^{\text{clip}} = \min\left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right).$$

- Максимизируем среднее по токенам каждого ответа, затем по группе.
- Добавляем KL-штраф относительно π_{ref} отдельно от наград группы.
- π^{old} фиксируем на цикл обновлений; π_{ref} задает опорное поведение.

Все ответы верны. Все преимущества равны нулю.



Выбор запросов и информативность группы

Что позволяет постановка МППР

- Выбирать запросы и повторять запуски из $s_0 = (x, \emptyset)$.
- Менять распределение задач $\mu(x)$ и число ответов K .
- Исследовать продолжения из фиксированного префикса.

Изменение μ меняет и цель обучения:

$$J_\mu(\theta) = \mathbb{E}_{x \sim \mu, y \sim \pi_\theta(\cdot|x)}[R(x, y)].$$

В базовом GRPO группа начинается с полного запроса; ветвление из префикса требует отдельной схемы обучения.

Когда нет сигнала от награды

$$(R_1, \dots, R_K) = (1, \dots, 1)$$

$$\text{или } (0, \dots, 0) \Rightarrow \hat{A}_i = 0.$$

- Нужны различающиеся результаты внутри группы.
- Больше ответов увеличивает стоимость генерации.
- Одинаковый итоговый вес слабо различает вклад отдельных токенов.

08

Задачи и среды для LLM

GSM8K: задача на рассуждение

Условие

- В книге 120 страниц.
- Вчера прочитано 12, сегодня вдвое больше.
- Завтра нужно прочитать половину остатка.

Сколько страниц прочитать завтра?

Решение

Сегодня: $12 \cdot 2 = 24$.

Осталось: $120 - 12 - 24 = 84$.

Завтра: $84/2 = \boxed{42}$.

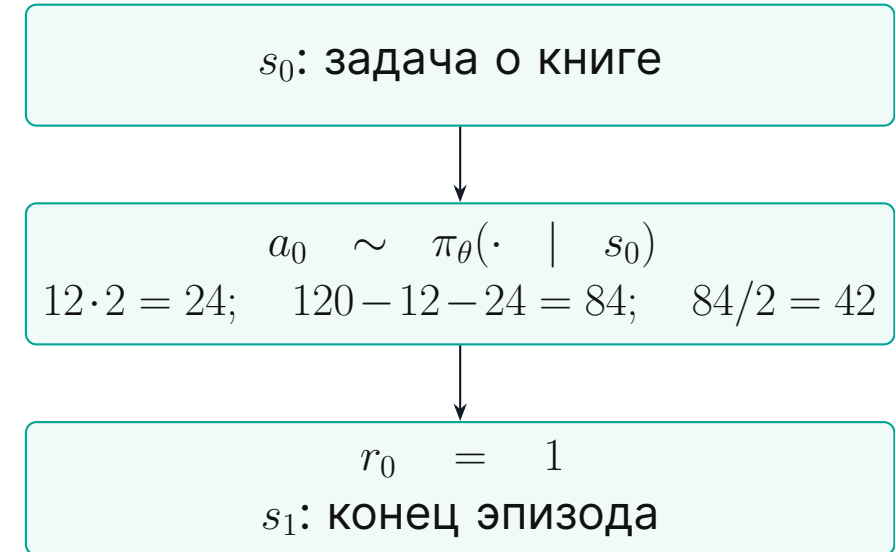
Награда 1, если итоговый ответ равен 42;
иначе 0.

Решение задачи как одношаговый MDP

- Состояние $s_0 = x$: условие задачи.
- Действие $a_0 = y$: полное решение с итоговым ответом.
- Награда $r_0 = R(x, y)$: проверка итогового числа.
- После ответа эпизод завершен: $G_0 = r_0$.

Один шаг среды — весь ответ.

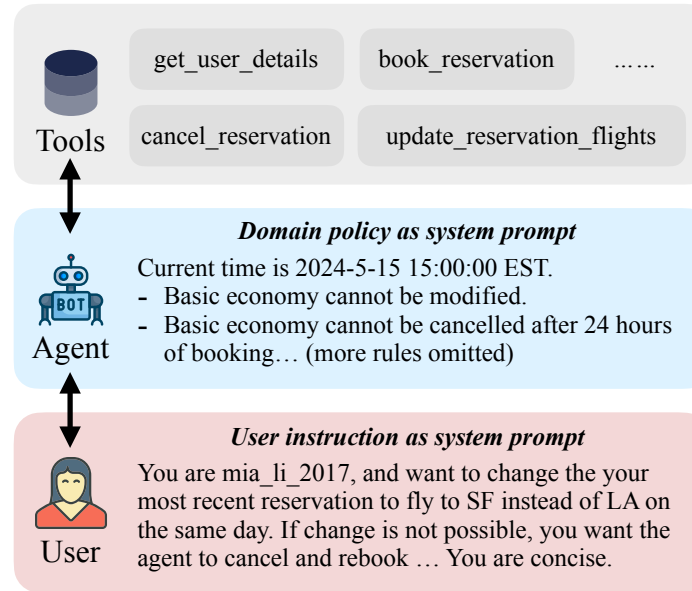
Модель генерирует его токен за токеном.



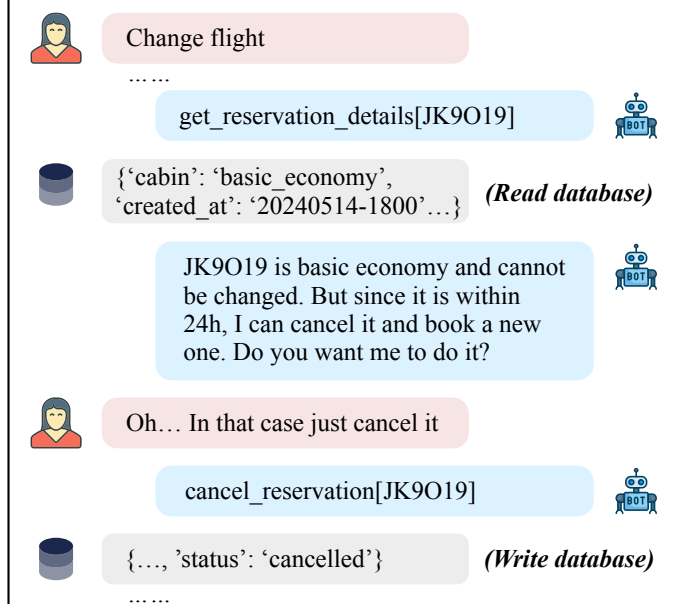
τ -bench: диалог и инструменты

- Агент общается с клиентом и вызывает API, соблюдая правила сервиса.
- Успех проверяем по итоговому состоянию базы данных.

(a) τ -bench setup

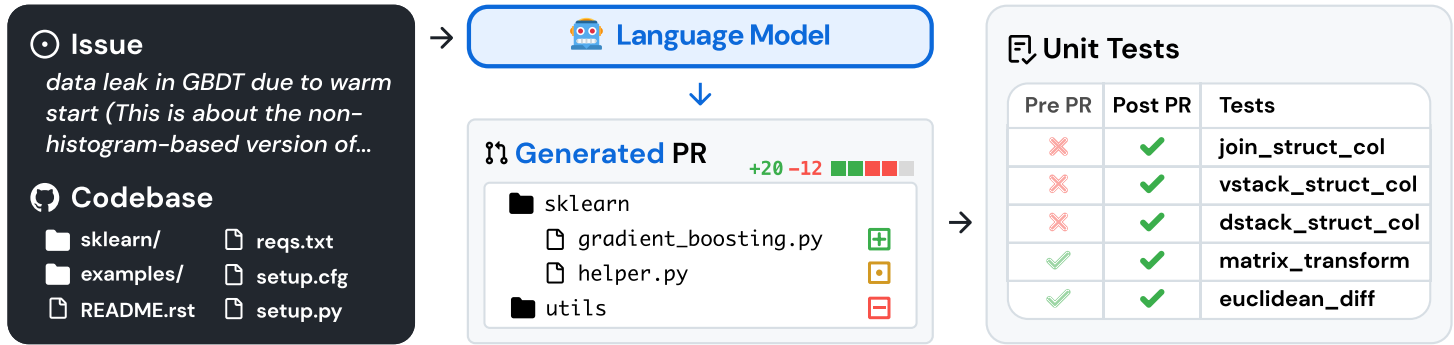


(b) Example trajectory in τ -airline



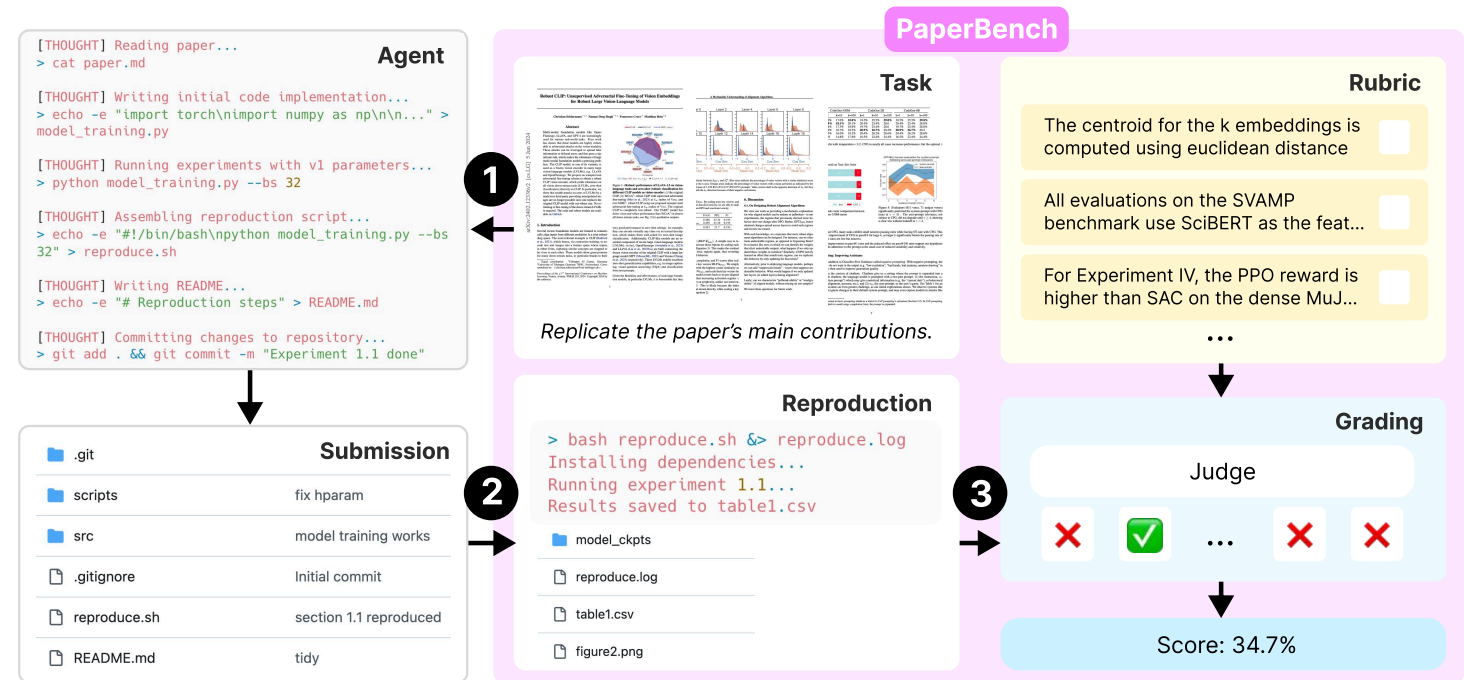
SWE-bench: исправление кода

- Даны описание проблемы и репозиторий; агент готовит патч.
- Исправление проверяем тестами, включая проверку прежней функциональности.



PaperBench: воспроизведение исследования

- По статье агент пишет код и проводит эксперименты.
- Результат оцениваем по критериям воспроизведения работы.



RL по полным траекториям взаимодействия

- Один эпизод — решение задачи за несколько действий.
- Агент получает ответы инструментов и выбирает следующий шаг.
- Оцениваем результат эпизода и обновляем стратегию по траектории.

Пример: SkyRL обучает агента работать с репозиторием через OpenHands.

Качество проверяем на отдельных задачах, например SWE-bench.



Вопросы?